

Versión 1.0.5 — Versión consolidada

Agrupar en una sola versión todo lo publicado después de 1.0.4 (antes 1.0.5 a 1.0.11): serie Serverless Edge, cuentas y cookies, tema blanco, tienda, banderas España / México, sitio en español, términos por región y el plan pendiente de i18n y CMS headless.

VERSIÓN 1.0.5 · staging	FECHAS 2026-10-03 → 2026-10-05	COMMITTS b2af7c1 ... 59a81ca · 8 commits	CAMBIOS 111 archivos, +7.419 / -2.930
-----------------------------------	--	--	---

1 Resumen

Entre el 3 y el 5 de octubre se publicaron siete versiones seguidas (1.0.5 a 1.0.11). Varias deshacían parte de la anterior: Google Translate con cuatro banderas entró en 1.0.6 y salió en 1.0.10; USD, GBP e ILS entraron y salieron de la tienda. Además, el documento 5 de seguridad ya tenía reservada la versión 1.0.6, que el readme usaba para otra cosa. Esta consolidación deja **una sola versión 1.0.5 con el estado final** del sitio y libera 1.0.6 para las medidas de seguridad.

Solo se reenumera la documentación. El historial de Git no se reescribe: los mensajes de los commits antiguos siguen diciendo «v1.0.6» ... «v1.0.11». La tabla del §2 relaciona cada commit con esta versión.

Revisión legal. Las cláusulas de México y la sección RAEE de España (§10) son un borrador redactado a partir de la normativa citada. Deben revisarlas un abogado antes de publicar, igual que los marcadores [...] del vendedor (incluido [Nº de registro RII-AEE]).

2 Historial consolidado

Antes	Commit	Contenido	Estado en 1.0.5
1.0.5	b2af7c1	Serie Serverless Edge: pasos 06–09 y cruces	Se mantiene (§4)
1.0.6	(en 1.0.7)	Banderas de EE. UU., España, México y Países Bajos con Google Translate; WhatsApp solo España	Banderas sustituidas en §8; WhatsApp se mantiene
1.0.7	b4372e9	Acceso / registro y consentimiento de cookies; idiomas ES ↔ EN	Pop-ups se mantienen (§5); idiomas sustituidos
1.0.8	f92487d	Tema blanco, paleta de cuatro colores; se retira ILS	Se mantiene (§6)
1.0.9	297bbac , c4d1ee1	Filtro en acordeón, tarjetas compactas, edición especial	Se mantiene (§7)
1.0.10	7671203	Solo España / México, divisa por bandera, geolocalización; sitio en español	Se mantiene (§8, §9)
1.0.11	59a81ca	Términos por región; plan de i18n y CMS headless; script DeepL	Se mantiene (§10–15)

3 Archivos (diferencia neta con 1.0.4)

Archivo o carpeta	Líneas	Cambio
js/layout.jsx	+411 / -38 · 754 en total	Banderas España / México, useLocale() , geolocalización, pop-ups de acceso y cookies, textos en español.
js/tienda.jsx	+137 / -49 · 739	CURRENCIES EUR / MXN, useCurrency() sigue a la bandera, CurrencyBadge , ProductTile , ProductRow , stock.

<code>js/blog.jsx</code>	+430 / -244 · 1.110	Pasos 06–09 de Serverless Edge, zona <code>platform</code> , todo el panel y los metadatos traducidos, fechas <code>es-ES</code> , <code>slugify</code> sin tildes.
<code>blog/serverless/</code> (13)	+1.787 / -38	Cuatro artículos nuevos y traducción del resto.
<code>blog/core</code> , <code>search</code> , <code>security</code> , <code>workflows</code> (32)	+1.806 / -1.864	Traducción al español (<code>lang="es"</code> , título, descripción, contenido).
<code>pages/components/</code> (41)	+618 / -113	Tienda con acordeón y edición especial; 40 fichas regeneradas.
<code>pages/terminos/index.html</code>	+341 / -143 · 419	Cláusulas por región con pestañas.
<code>public/db/*.json</code>	+438 / -56	Grupos de categorías, 13 categorías, campo <code>edition</code> , 10 productos de edición especial.
<code>css/site.css</code>	+136 / -19 · 194	Paleta de cuatro colores y mapeo de clases de Tailwind.
<code>index.html</code> , <code>whatsapp-chat-snippet.html</code>	+186 / -166	Widget de WhatsApp solo España; textos y comentarios en español.
<code>tools/il18n/deepl-prefill.mjs</code> NUEVO	+143	Pre-traducción con DeepL (pendiente de uso).
<code>readme.md</code> , <code>public/db/README.md</code>	+319 / -185	En español; versiones consolidadas.

4 Blog · Serverless Edge

- La serie pasa de 5 a 9 pasos en `BLOG_TYPES` : 06 Tiempo real (WebSockets y Durable Objects), 07 IA en el edge (Workers AI, Vectorize, AI Gateway), 08 Imágenes y multimedia (R2, transformaciones, Stream), 09 Desplegar, probar y lanzar (Wrangler, Vitest, despliegues graduales).
- Tres cruces sobre la oferta de plataforma: PaaS de integración, Edge × IaaS (híbrido) y SaaS multiinquilino con medición.
- Nueva zona `platform` para el paso de despliegue; los artículos se encadenan con «Siguiente paso»: 05 → 06 → 07 → 08 → 09 → visión general.

5 Cuentas y cookies

Pieza	Comportamiento	Almacenamiento
<code>AuthModal</code> , <code>auth</code>	Pop-up con pestañas «Iniciar sesión» / «Crear cuenta» desde la cabecera y el menú móvil; foco atrapado, Escape cierra.	<code>lm-users</code> (SHA-256 con sal), <code>lm-session</code> — solo staging, en el navegador
<code>CookieConsent</code>	Aparece en la primera visita; «Aceptar todas» / «Solo necesarias». Las analíticas solo deben cargarse si <code>hasCookieConsent()</code> .	<code>lm-cookies</code> , <code>window.LM_COOKIES</code>

Las cuentas de staging no tienen servidor: cualquiera con acceso al navegador puede leerlas. Su sustitución por Supabase Auth es parte de 1.0.6 (documento 5, §8).

6 Diseño

Tema blanco con cuatro colores definidos como tokens al principio de `css/site.css` : blanco `#ffffff` (fondo), azul `#2563eb` (botones, enlaces, acentos), azul oscuro `#0a192f` (encabezados, texto, banner superior, secciones oscuras) y gris `#64748b` (pies de texto). El final del archivo asigna las clases de Tailwind usadas a esos colores.

7 Tienda

- Filtro en acordeón:** barra lateral en escritorio y desplegable «Filtros» en móvil, con 4 grupos (Componentes del PC, Redes, Equipo básico, Herramientas) y 13 categorías por tipo de pieza; opción «Solo productos en stock». Los contadores muestran lo que devolvería cada categoría.
- Tarjetas compactas** (`ProductTile`): 4 × 2 en escritorio, 2 × 4 en móvil, 8 por página con paginación.

- **Edición especial** (`edition: "especial"`): lista sin foto en filas de una línea (`ProductRow`), borde con el color del stock: 1 unidad granate, 2 unidades azul, 3+ verde, agotado gris.
- El carrito nunca guarda más unidades que el stock.

8 Banderas, país y divisa

Paso	Qué hace
1 · Bandera guardada	Si el visitante pulsó una bandera (<code>lm-locale</code>), se usa siempre.
2 · Zona horaria	Sin bandera: <code>America/*</code> y <code>Pacific/*</code> → México; el resto → España. Instantáneo y sin red.
3 · Geolocalización	Una consulta a GeoJS (<code>get.geojs.io</code>). ES o MX se usan tal cual; cualquier otro país va al más cercano por distancia a Madrid o Ciudad de México. Se guarda en <code>lm-geo</code> y no se repite.
Divisa	España → EUR, México → MXN (tipo del BCE vía Frankfurter, con respaldo). El cobro sigue en EUR; los pesos se muestran con «≈».

El cambio de bandera se propaga con el evento `lm-locale` y entre pestañas con `storage` , sin recargar. Se borra la cookie `googtrans` que dejaba la versión con Google Translate.

9 Sitio en español

- Blog completo: panel, 38 artículos, series, tipos, zonas, facetas y etiquetas (con un glosario común para que los filtros sigan agrupando igual).
- Navegación (Desarrollo, Componentes, Comunidad), botones (REGISTRO), opciones del formulario de contacto, textos de los pop-ups.
- Fechas del blog en `es-ES` ; `slugify` elimina tildes, así que los anclajes de los encabezados quedan legibles.
- Se mantienen en inglés los nombres de carpeta y los slugs (para no romper enlaces), los comentarios del código y la opción English del widget de WhatsApp.

10 Términos por región

10.1 Cómo funciona

pages/terminos/index.html

```
const region = useLocale().country;           // 'ES' o 'MX', la bandera de la cabecera
const sections = sectionsFor(region);          // filtra SECTIONS por `regions` y numera 01, 02...
CLAUSES.garantia = { ES: GarantiaES, MX: GarantiaMX } // cláusula distinta por región
CLAUSES.vendedor = Vendedor                   // cláusula común (recibe `region`)
```

- **Pestañas = bandera.** Pulsar «México» en el índice llama a `chooseLocale()` : cambia también la bandera de la cabecera y la divisa de la tienda. Abrir los términos con la bandera de México muestra directamente la pestaña México.
- **Mismos anclajes.** Los ids (`#devoluciones` , `#garantia` , `#envios` , `#precios`) existen en ambas regiones, así que los enlaces de las fichas de producto y del carrito siguen funcionando. Al cargar, la página baja al anclaje cuando la cláusula ya está renderizada.
- **Accesibilidad.** `role="tablist"` / `tab` / `tabpanel` y `aria-selected` .

10.2 Índice por región

#	España · política estándar española, electrónica	México · LFPC
01	Datos del vendedor (LSSI-CE, Ley 34/2002)	Datos del vendedor (art. 76 bis LFPC)
02	Objeto y aceptación	Objeto y aceptación
03	Productos	Productos + información en español y certificación de productos eléctricos
04	Precios y divisas (EUR, IVA incluido)	Precios y divisas (MXN orientativo, total en moneda nacional antes del pago)
05	Pedidos y pago	Pedidos y pago (confirmación en EUR y MXN)
06	Envíos y recogida (España, recogida en Madrid)	Envíos a México y aduanas

07	Devoluciones: desistimiento 14 días (TRLGDCU) + ampliación a 30	Revocación 5 días hábiles (art. 56 LFPC) + ampliación a 30
08	Garantía legal de conformidad: 3 años	Garantía de 3 años (mínimo legal 90 días, art. 77 LFPC)
09	NUEVO Residuos electrónicos y pilas (RAEE, RD 110/2015, uno por uno)	—
10 / 09	Protección de datos (RGPD, LOPDGDD, AEPD)	Aviso de privacidad, derechos ARCO
11 / 10	Ley aplicable (OMIC, Junta Arbitral de Consumo de Madrid)	Ley aplicable (PROFECO, Concilianet)
12 / 11	Formulario de desistimiento	Aviso de revocación

El cobro sigue en EUR también para México. La LFPC exige informar del precio total en moneda nacional; la cláusula lo cubre con la confirmación en MXN antes del pago, pero cobrar en pesos (pasarela local) es una decisión de negocio pendiente.

11 Opción A — Vite + react-i18next **PENDIENTE**

Mantiene el sitio como SPA de React, pero con un paso de build: Vite compila el JSX (adiós a Babel en el navegador) y react-i18next carga los textos por página.

11.1 Estructura

```
locales/
  es/ common.json inicio.json tienda.json terminos.json blog.json blog-security-rate-limiting.json ...
  en/ ...
src/i18n.ts // relleno por DeepL, revisado en el TMS
```

- **Un namespace por página** (`terminos`, `tienda`, un namespace por artículo del blog) más `common` para cabecera, pie, cookies y acceso.
- **Carga diferida:** cada namespace es un `import()` dinámico; Vite lo convierte en un chunk propio que solo se descarga en la página que lo usa.
- Los términos por región se modelan como claves dentro del namespace: `terminos:es.garantia.pl`, `terminos:mx.garantia.pl`.

src/i18n.ts

```
import i18n from 'i18next';
import { initReactI18next } from 'react-i18next';
import resourcesToBackend from 'i18next-resources-to-backend';

i18n.use(initReactI18next)
  .use(resourcesToBackend((lng, ns) => import(`../locales/${lng}/${ns}.json`))) // un chunk por namespace
  .init({ lng: 'es', fallbackLng: 'es', ns: ['common'], defaultNS: 'common',
    interpolation: { escapeValue: false }, react: { useSuspense: true } });

// en cada página: const { t } = useTranslation('terminos'); t('es.garantia.pl')
```

11.2 Límites

- Sigue siendo renderizado en cliente: el HTML inicial sigue vacío para los rastreadores (ver el artículo «SSR y generación estática» del blog). Se puede prerenderizar con `vite-plugin-ssr` / `vike`, pero eso ya se acerca a la opción B.
- Las URLs no cambian de idioma; un segundo idioma necesitaría `?lng=` o rutas propias y `hreflang` a mano.
- Esfuerzo estimado: extraer ~1.500–2.000 cadenas a JSON y cambiar cada texto por `t()`; migración página a página posible.

12 Opción B — Next.js + next-intl + generación estática + /es/ **PENDIENTE**

Pasa a Next.js (App Router) con exportación estática: cada página se genera en build como HTML completo bajo un prefijo de idioma, lo que además resuelve el HTML vacío del sitio actual.

```

app/[locale]/layout.tsx // NextIntlClientProvider, <html lang={locale}>
app/[locale]/pages/terminos/page.tsx // /es/pages/terminos/
app/[locale]/blog/[series]/[slug]/page.tsx // /es/blog/security/rate-limiting/
i18n/routing.ts i18n/request.ts messages/es/*.json

```

i18n/routing.ts · next.config.mjs

```

export const routing = defineRouting({ locales: ['es'], defaultLocale: 'es', localePrefix: 'always' }); // 'en'
cuando se traduzca

// en cada page.tsx / layout.tsx
export function generateStaticParams() { return routing.locales.map(locale => ({ locale })); }
setRequestLocale(locale); // habilita el renderizado estático

// next.config.mjs
export default withNextIntl({ output: 'export', trailingSlash: true, images: { unoptimized: true } });

```

- **Mensajes por página:** un archivo por namespace en `messages/es/`, combinados en `i18n/request.ts`; cada página solo pasa al cliente los namespaces que usa.
- **Exportación estática sin middleware:** con `output: 'export'` el middleware de next-intl no se ejecuta, así que / necesita una página que redirija a `/es/`. Las URLs actuales (`/pages/...`, `/blog/...`) deben redirigir a `/es/...`: reglas de redirección de Cloudflare (301 reales) o la lista `moved` de `404.html`.
- **SEO:** `<link rel="alternate" hreflang>` y canónicas por idioma generadas desde `routing`; sitemap generado en build.
- **Región ≠ idioma:** España y México comparten `/es/`; la bandera sigue guardándose en el navegador (o en una cookie) y elige divisa y cláusulas. Rutas `/es-mx/` solo si algún día cambia el texto, no solo la divisa.
- Esfuerzo estimado: mayor que A (reescritura a componentes de Next, datos del blog y de la tienda en build), pero entrega HTML estático rastreable.

13TMS y pre-traducción con DeepL

13.1 Tolgee o Crowdin (plan gratuito)

TMS	Encaja porque	A tener en cuenta
Tolgee	Integración directa con react-i18next (<code>@tolgee/i18next</code>) y edición en contexto sobre la propia página; CLI para subir y bajar JSON; código abierto, se puede autoalojar.	Revisar los límites actuales del plan gratuito en la nube (claves, puestos). Autoalojado no tiene esos límites.
Crowdin	Sincronización con GitHub mediante <code>crowdin.yml</code> : lee <code>locales/es/*.json</code> y abre pull requests con las traducciones; memoria de traducción y glosario.	El plan gratuito está pensado sobre todo para proyectos públicos; comprobar el límite de proyectos privados y palabras.

crowdin.yml (si se elige Crowdin)

```

files:
  - source: /locales/es/*.json
    translation: /locales/%two_letters_code%/%original_file_name%

```

13.2 Flujo propuesto

1. Se escribe o edita el texto en español (`locales/es/<página>.json`) en un pull request.
2. `node tools/i18n/deepl-prefill.mjs --target en` rellena solo las claves que faltan y las anota en `locales/en/_prefill.json`.
3. El TMS importa las traducciones como *sin revisar*; una persona las corrige y aprueba.
4. El TMS devuelve los JSON aprobados al repositorio (pull request de Crowdin o `tolgee pull`) y el build los publica.

13.3 tools/i18n/deepl-prefill.mjs

- Solo rellena claves vacías o ausentes; nunca sobrescribe una traducción revisada. Mantiene el orden de claves del archivo fuente.
- Protege `{{name}}` (react-i18next), `{name}` (next-intl / ICU) y los términos de marca (`Lameyer`, `MC-SE`, `Modular Core`, `Serverless Edge`, más `--keep`) con etiquetas `<x>` que DeepL no traduce.

- `--dry-run` cuenta claves y caracteres sin llamar a la API; útil para no pasar del plan gratuito de DeepL API (500.000 caracteres al mes). Clave `:fx` → endpoint gratuito automáticamente.
- Probado contra un servidor DeepL simulado: placeholders y marcas intactos, claves existentes conservadas, segunda ejecución sin cambios.

14Comparativa

Criterio	A · Vite + react-i18next	B · Next.js + next-intl
HTML inicial para buscadores	Vacío (SPA) salvo prerenderizado extra	Completo, generado en build
Rutas de idioma	No incluidas	<code>/es/...</code> nativo, <code>hreflang</code>
Carga de textos	Namespace por página, chunk diferido	Namespace por página, solo lo que usa cada ruta
Esfuerzo	Menor, migración gradual	Mayor, cambio de framework y de URLs
Alojamiento	GitHub Pages igual que hoy	GitHub Pages / Cloudflare Pages (exportación estática)

Recomendación (pendiente de decisión). B como destino, porque el blog depende de que los buscadores lean el contenido y las rutas `/es/` preparan un segundo idioma. A solo si hace falta traducir pronto sin cambiar de framework. En ambos casos el TMS y el script de DeepL son los mismos.

15Pendiente

#	Tarea	Prioridad
1	Elegir opción A o B	BLOQUEA
2	Elegir TMS (Tolgee o Crowdin) y crear el proyecto con origen <code>es</code>	ALTA
3	Extraer las cadenas a <code>locales/es/<página>.json</code> (empezar por <code>common</code> y <code>terminos</code>)	ALTA
4	Clave de DeepL API Free como secreto de CI (<code>DEEPL_API_KEY</code>)	MEDIA
5	Solo B: redirecciones 301 de las URLs actuales a <code>/es/...</code> y sitemap con <code>hreflang</code>	MEDIA
6	Revisión legal de las cláusulas de México y de RAEE; número RII-AEE	ALTA
7	Decidir si los pedidos de México se cobran en MXN	BAJA